

HBaseMQ: A Distributed Message Queuing System on Clouds with HBase

Chen Zhang and Xue Liu

School of Computer Science, McGill University, Montreal, Canada

Email: {czhang,xueliu}@cs.mcgill.ca

Abstract—Message queuing systems can be used to support a plethora of fundamental services in distributed systems. This paper presents HBaseMQ, the first distributed message queuing system based on bare-bones HBase. HBaseMQ directly inherits HBase's properties such as scalability and fault tolerance, enabling HBase users to rapidly instantiate a message queuing system with no extra program deployment or modification to HBase. As a result, HBaseMQ effectively enhances the data processing capability of an existing HBase installation. HBaseMQ supports reliable and total order message delivery with “at least once” or “at most once” message delivery guarantees with no limit on message size. Furthermore, HBaseMQ provides atomicity, consistency, isolation and durability on any operation over messages.

Index Terms—Distributed message queuing system; HBase; Clouds;

I. INTRODUCTION

Message queuing systems allow asynchronous message communication between senders and receivers. Both senders and receivers are free to choose their own pace of message processing for optimized performance without worrying about message congestion at either side of the communication channel. Message queuing systems are not intended for use cases that require immediate responses and that lack of a response will prevent the application from continuing. Distributed message queuing systems emphasize on the usage scenarios that queues and operations on queues may be distributed for scalability and fault tolerance purposes in distributed computing environments such as clouds.

Many message queuing systems, proprietary/commercial and open source versions, exist with different target use cases and system requirements conforming to various message queuing protocols (e.g. AMQP [2], REST [3], XMPP [4], etc.). Proprietary message queuing systems are industrial strength products with rich functionalities and hard delivery guarantees (e.g. total order, “at most once”, etc.). Example systems are IBM's WebSphere MQ [8], Microsoft's MSMQ [9], Oracle's Advanced Queuing [10], etc. Different from these advanced queuing systems, Amazon provides a low-cost Simple Queue Service (SQS) [13] with very basic messaging operations (send/receive) and no guarantee on message delivery ordering and times. There are also many open source message queuing systems providing different levels of services, ranging from full-fledged functionalities (comparable to proprietary systems) to bare basic messaging needs. Example systems include ActiveMQ [1], RabbitMQ [12], etc.

As cloud computing grows popular, Hadoop [6] and the Hadoop database called “HBase” [7] are used by more and more applications. Many of those applications need a certain kind of message-based communication between distributed tasks for synchronization. However, existing message queuing systems mentioned above are often either too specialized/sophisticated (e.g., full-fledged message queuing systems) and costly to deploy on-demand (e.g., the proprietary systems by Microsoft, etc.) or too simple to provide the desired level of message delivery guarantees. The bottom line is, in order to use message queuing, some queuing system must be deployed in addition to existing Hadoop/HBase installation, which creates extra complexity and overhead in managing application scalability and faults. One simple example of this added complexity is that, suppose an extra queuing system is used, applications must be able to handle faults caused by both/either the queuing system and/or Hadoop at the same time.

In other words, Hadoop/HBase lacks advanced message queuing functionality without having to deploy any extra queuing systems and the current technology with other message queuing systems does not have incorporated functionality and properties the same as those provided by Hadoop/HBase. Under this general background, this paper presents HBaseMQ, the first distributed message queuing system implemented as a light-weight client library to HBase. HBaseMQ directly makes use of the underlying bare-bones HBase without adding any extra system components. Furthermore, HBaseMQ supports reliable and total order message delivery with “at least once” or “at most once” message delivery guarantees. Users can also send messages with arbitrary size and perform customized search operations to filter messages. Operations on messages are guaranteed to satisfy database level ACID requirements, namely, atomicity, consistency, isolation and durability.

The major challenges of building HBaseMQ are as follows: 1) Only make use of passive HBase tables to guarantee message ordering while HBase tables provide no guarantee that the sequence of row appearance is the same as the sequence of row insertion, and there is no extra system component deployed to act as a message ordering manager. As a result, we must design a protocol to “deliver” acknowledgement of message sent ordering to participating client applications such that clients can rest assured that their messages are accepted by the system and will be delivered in order; 2) Guarantee transactional message operation over HBase tables that do not

support transactional read/write sequences. This is necessary for the “at least once” and “at most once” delivery guarantees and for maintaining message integrity (ACID requirements).

This paper describes the details of HBaseMQ design and implementation addressing the above challenges. Performance evaluations are conducted to show the effectiveness of HBaseMQ messaging protocol and quantify the cost of our approach. The remainder of the paper is structured as follows: Section II discusses related work. Section III describes the design and implementation of HBaseMQ. Section IV reports results on performance evaluations and Section V concludes.

II. RELATED WORK

HBaseMQ is the first message queuing system built as a client library to HBase, aiming to help existing HBase users deploy a message queuing system with minimum effort. The message delivery guarantees and message operation functionalities provided by HBaseMQ are normally only available on full-fledged message queuing systems (for example, [8], [9], [10], [1]) that require installing and maintaining a standalone message system. Yet the simplicity of HBaseMQ puts itself in the light-weight system category. There is no existing light weight system on Hadoop/HBase that provides comparable features without requiring to install and run extra system software. We have to point out that it is definitely possible to implement a light-weight message queuing system using some of the state-of-the-art queuing systems just mentioned. However, the focus of HBaseMQ is for users with an existing Hadoop/HBase installation to enable message queuing with some important features (such as “at most once” delivery guarantee and transactional message data consistency) without the need to add any extra software. The benefit of doing so is multi-fold. For example, from the system administrator’s point of view, no extra work needs to be done to deploy and maintain any extra software; from the Hadoop/HBase users point of view, the learning curve to use HBaseMQ is flat because it is not so much different from using existing HBase API; HBaseMQ inherits many well-tested cloud-ready properties from Hadoop/HBase (because it is just a light-weight client library and does not change the system properties of Hadoop/HBase) such as scalability and fault tolerance, etc.

Among message queuing systems that are cloud-ready (without requiring any system installation from users), Amazon SQS [13] is the most relevant system to compare with. SQS emphasizes on scalability and availability of messages in clouds. To do that, it replicates messages at distributed locations that are not sync-ed atomically (some replicas may become outdated). As a result, it does not give any guarantee to users that their applications will be redirected to a replica that holds previously sent messages in a timely manner. A phenomenon SQS users often witness is that the applications need to poll for previously sent messages with indeterministic latency before being notified the existence of such messages. Furthermore, only “at least once” delivery is promised and there is no guarantee on the order of message delivery either. The message operations are also limited to

simple message send/receive based on message IDs with no customized search functionality to filter messages using user-defined meta-data. In comparison to SQS, HBaseMQ achieves stronger message delivery guarantees and supports richer message operations with ACID properties. We argue that HBase users with advanced message queuing needs should consider using HBaseMQ over SQS.

III. HBASEMQ

In this Section, we describe the details of HBaseMQ design and implementation. The general idea is to create several HBase tables to manage messages using a novel protocol.

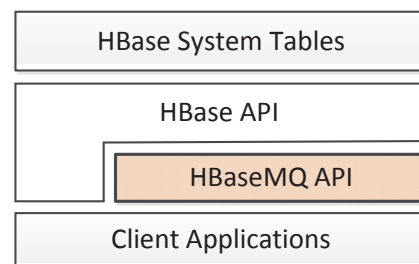


Fig. 1. HBaseMQ architecture. HBaseMQ is a light-weight client library implemented by using the HBase API. Client applications send/receive messages by using the HBaseMQ API, and perform other HBase related operations by using the normal HBase API. Each HBase system table represents a message queue to hold client messages.

A. Design Overview

In HBaseMQ, applications that need to send/receive messages use the HBaseMQ library to interact with HBase instead of using the standard HBase API, as illustrated in Figure 1. Each message queue is represented by an HBase “system table” named after the queue. A message is coded as a row in the HBase “system table”.

Table I shows an example system table for a queue. The table contains several rows. Rows with “UUIDs” as row keys are messages put into the queue. The “messageBody” column stores the contents of the message or a pointer to files as long as the senders/receivers know how to fetch the files. The “type” column stores customized data type information such that the receivers of the message know how to interpret the byte array message body. The “deliveryGuarantee” column specifies whether the message should be delivered “at most once” or “at least once”. The “timestamp” columns represent different unique timestamps. The “Y” character in each timestamp column is a mere indicator that the column is not empty. There is only one row in the same queue table that contains non-empty value under one particular timestamp column (timestamps are unique within a queue). This provides convenience to rapidly pinpoint a message by its timestamp. As a result, a typical queue table has a large number of timestamp columns. This is perfectly fine with HBase and does not waste any physical disk space because HBase is a key-value store internally. Rows with client IDs as row keys are clients that are registered to the queue to send/receive messages. The “lastReceivedMessageTimestamp” column records

TABLE I
AN EXAMPLE HBASEMQ SYSTEM TABLE FOR A QUEUE.

Row Key	messageBody	type	deliveryGuarantee	timestamp1	timestamp2	lastReceivedMessageTimestamp	counter
UUID1	contents1	string	at most once	Y			
UUID2	contents2	number	at least once				
clientID1				delivered		timestamp1	
clientID2				delivered		timestamp1	
timestamp							timestamp2
timeout					Y		

the most recently received message timestamp. A client should only receive messages according to the total order specified by message timestamps. The “timestamp” row and the “counter” column of that row are used for dispensing unique timestamps for messages in the queue. The “timeout” row is used for marking if a message timestamp has timeout. If a timestamp column is marked as timeout in the “timeout” row (containing value “Y”), the particular message timestamp becomes invalid (See details in fault tolerance mechanism discussion later) and the message with that timestamp will be discarded. Notice the sparse nature of the queue table: each type of rows makes use of a different set of columns and contains no value in other columns. We can then make use of HBase’s fast column-based search to scan the queue table for different types of rows.

To send a message to a queue, the user needs to specify/create a queue table and insert the message as a row to the table. The message row must contain certain columns (such as a column for the message timestamp, and for the type of delivery guarantee needed, either “at most once” or “at least once”) required by the system and can optionally contain any number of other columns to store message meta-data so that the user can conduct customized queries on those meta-data. To get a message from a queue, the user needs to visit the corresponding queue table, read the message and perform certain operations to mark the delivery status of the message according to HBaseMQ protocol.

B. Unique ID and Timestamp

HBaseMQ messages make use of three globally unique IDs that do not need to be well-ordered, and a timestamp that must be unique and strictly ordered within a queue. The first ID is the queue ID, used as the name for a queue and the corresponding queue table. The second is the client ID, used as the row key of a registered client to a certain queue. The third is the message ID, used as the row key of a message row at a queue table. The timestamp is unique and used to indicate the message ordering within a queue. It is stored as a separate column with non-empty value in the message row. The queue ID and client ID are assumed to be predefined and unique because they should be static variable on a fixed problem scenario. The client ID should be kept constant also because of fault tolerance purposes: if a client crashes and restarts, it should still be able to resume its previous identity and message history. To the contrary, the message ID is a variant for every new message and is generated as a globally unique UUID in java. The timestamp for each message is

obtained by using the incrementColumnValue method on the “timestamp” row of a queue table. We adopt this timestamp generation mechanism because we want to use 100% HBase functionality without using external timestamp issuers. Other more efficient timestamp mechanisms [11] can be used if the current mechanism becomes the system bottleneck. More specifically about getting the timestamp, when the HBase atomic incrementColumnValue method is used on the column “counter” of row “timestamp”, the data value in the column is incremented by “1” and the new value is returned as a strictly incremental timestamp associated with this particular queue. Note that timestamps are bound to each queue and are not guaranteed to be unique or strictly-ordered across different queues.

C. Message Queuing Protocol

First, we define several key terms used in the descriptions of the protocol. A “sent message” to a queue is denoted by a message row in a queue table having a non-empty timestamp column (with value “Y”). A “delivered message” to a client is denoted by a client row in a queue table having a non-empty timestamp column (with value “delivered”) corresponding to the delivered message. The next message of a client to receive has a timestamp equals to the client’s most recently received message timestamp + “1”. A message is deemed as “timeout” if the timestamp column (corresponding to the message) of the “timeout” row in a queue table is marked as “Y”. A message that has timeout cannot be delivered to any client. Now we will describe the protocol of sending and receiving messages in sequence.

Sending Messages After a client registers itself in a queue, it can start to send messages to the queue. When sending message, a client must finish inserting the message row into the queue table before obtaining the message timestamp. Then the client uses an atomic HBase checkAndPut method to write “Y” to the message row under the timestamp column, if the value of the timestamp column in the “timeout” row is empty, i.e. not marked as timeout. An atomic HBase checkAndPut() method is used here to make sure that the message is not being or has not been marked as timeout by other waiting clients (all the clients registered to the same queue may wait for the message) while the message sending client is trying to edit the message row. Only after “Y” is successfully written to the timestamp column in the message row can the message be declared a “sent message”.

Receiving Messages To receive a message, a client first gets

the timestamp under column “lastReceivedMessageTimestamp” as T_0 (if non-exist then use the smallest timestamp by scanning timestamp columnfamily for all rows), scan rows with non-empty column T_0+1 . If no rows exist, wait for a period that equals to the timeout threshold and recheck. If after the waiting there are still no existing rows, mark the timestamp T_0+1 as timeout by calling the atomic checkAndPut method on “timeout” row under the column T_0+1 . If marking column T_0+1 as timeout succeeds, then no clients can send/receive a message with that timestamp, retry scanning with timestamp T_0+2 ; if not, which means the message is “sent” right before the client tries to mark it as timeout, consume the message and update T_0 . If the result set of the scan for rows under column T_0+1 contains non-empty rows, check if the message is marked as timeout in the “timeout” row or has already been delivered to the client itself. If the message has timeout, delete it, retry scanning with timestamp T_0+2 ; if the results contain the client itself as “delivered”, retry scanning with timestamp T_0+2 ; otherwise consume the message and update T_0 .

Key Explanations After a client registers itself in a queue, it can start to receive messages from the queue. By scanning the queue table using the next expected timestamp, a client can rapidly pinpoint the message if it has not timeout. For messages appearing in the queue table within the timeout threshold, the delivery guarantee is checked before the client consumes the message. HBaseMQ checks if the client has taken the message before by looking at the timestamp column of the client row. If the delivery guarantee is “at most once” and the timestamp column value is “delivered”, the corresponding message will not be delivered again. The sequence of operations, i.e. first save a durable HDFS copy of the message (in user-predefined HDFS paths such that revived client after failure still knows where to access this HDFS copy), and then mark the message as “delivered” and consume the HDFS copy of the message in application, is very important for keeping the “at most once” guarantee. Otherwise, if the client first directly consumes the message and then marks it “delivered”, it is possible that the client may crash after message consumption but before marking it “delivered”. Then after the client revives, the same message will be delivered again mistakenly, breaking the “at most once” guarantee.

Note that if a message is marked as timeout, the client sending that message will normally retry with another timestamp until an acceptable timestamp is found. If eventually the client has to stop retrying, it will remove the message row from the queue table. However, if the client fails before being able to clean up message row, garbage data will be left untreated. In HBaseMQ, garbage data cleaning is done in a collaborative fashion by all clients that receive messages. If any client happens to discover unfinished cleanup jobs, it performs the remaining cleanup operations. Even if the original client itself is performing cleanup normally (but not yet finished), others can still check and help delete unwanted data.

Faults in HBaseMQ normally come from four different sources, i.e., server-side machine failures, network failures, client failures and suspected failures (also called “straggler-

s”). HBaseMQ relies on HBase and the underlying Hadoop Distributed File System (HDFS) to handle the first two kinds of failures. For client failures and stragglers, HBaseMQ by default uses a simple timeout mechanism: if a client has taken too long to finish sending a message, the message will be marked as timeout by other waiting clients. Because the time needed to finally mark a message as a “sent message” is only a simple write operation to the timestamp column, the time it takes should be near constant. Therefore a single global timeout threshold can be used. In fact, users can choose to use their own message failure handling mechanism. A checkAndPut method is always used to make sure a message is either sent or never sent, atomically.

HBaseMQ provides reliable total order message delivery with “at most once” or “at least once” guarantee. The “at least once” guarantee can always be satisfied because the data for sent messages are durable and reliable. “At most once” guarantee also holds because before a message is marked as “delivered”, only a durable local copy of the message is being saved by the HBaseMQ system under user-predefined paths, which can be done multiple times if interrupted by failures without affecting application logics that require the message contents. Only after a message is marked as “delivered”, will the real application logics at client side be initiated. Therefore, from a client’s point of view, a message is only delivered at most once to the application that needs the message.

HBaseMQ also guarantees ACID message operations, namely, Atomicity, Consistency, Isolation, and Durability. The Atomicity property is guaranteed by the atomic checkAndPut method of HBase when marking messages as “sent messages” by writing “Y” under the timestamp column of the messages. According to the protocol in receiving messages, only when a message is marked as “Y” under a valid (not timeout) timestamp will the message be deliverable to any client. Therefore, atomicity is satisfied. The Consistency property is satisfied because we assume that users only put valid data format (HBase table requires data in byte array format) into the tables. The Isolation property is satisfied because a message is only under concurrent read-only access (receiving) after it has been marked as a “sent message” by an atomic operation, while before being marked as “sent”, only a single sender can operate on the message. The Durability property is also satisfied because HBase provides a strong data consistency model for storing data in the underlying Hadoop Distributed File System (HDFS) which is assumed to be durable and reliable.

IV. PERFORMANCE EVALUATIONS

The general purpose of the performance evaluations is to quantify the cost of adopting the HBaseMQ system. Because HBaseMQ is the first system built on top of Hadoop/HBase without requiring to install and run any additional systems, currently there are no directly comparable solutions. HBaseMQ does not attempt to compete with full-fledged message queuing systems in performance, because those systems are specialized systems with bundles of optimizations

tailored to message queuing. Instead, HBaseMQ aims to provide rich functionalities for HBase users in a cloud-ready way (no extra system installation/configuration required from users). Therefore, we choose to perform mini-tests on our system components and also compare some performance with Amazon SQS, another cloud-ready message queuing system with very basic functionalities. We will show that HBaseMQ outperforms Amazon SQS in certain use cases. We use 20 Amazon EC2 instances in total (unless specified otherwise) to perform the tests. An HBase region server is deployed on all 20 machines. Clients are evenly distributed among 19 out of the 20 machines while the remaining 1 machine runs the HBase master. The ratio of client send/receive message is 1. To minimize EC2 performance variance, we perform each test 3 times and plot the average.

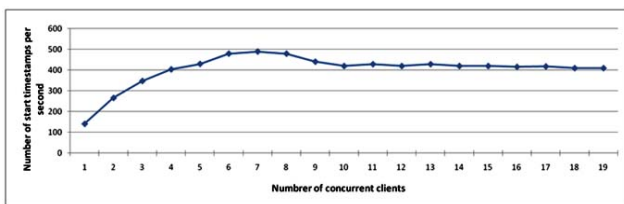


Fig. 2. Test1: Timestamp issuing mechanism.

Test1 is to measure the performance of our timestamp issuing mechanism and see how many timestamps our system can generate after saturation caused by an increasing number of concurrent clients. All clients are started at the same time for each test batch, one client at each machine instance. Each client keeps on requesting for new timestamps one after another and stops at the same time. Note that this test is specially performed using only one region server on a single EC2 instance without using the other 19 instances. It is because we are only using one HBase table data field to disseminate timestamps and therefore using additional region servers won't boost performance. The result is shown in Figure 2. As we can see, the timestamp issuing system saturates quickly, reaching about 400 timestamps per second, or about 34 million timestamps per day. We argue that it is sufficient for at least medium scale application usage scenarios.

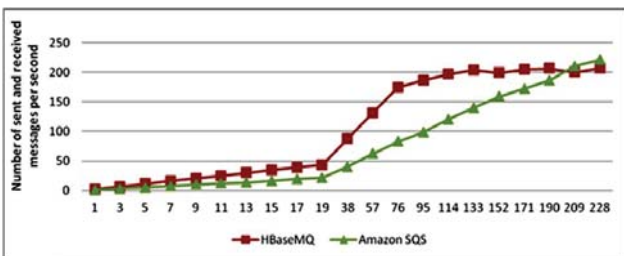


Fig. 3. Test2: Comparative performance of HBaseMQ vs Amazon SQS.

Test2 is to measure the comparative performance between HBaseMQ and SQS in terms of system throughput (number of successful sent and received messages per second) with an increasing number of concurrent clients. Again, all clients are started at the same time for each test batch, and we maintain

an even number of clients across 19 machine instances. All clients send messages to the same queue and receive the same number of messages from that queue as the number of messages sent (the ratio of send/receive is 1). Clients keep on sending/receiving and stop at the same time. As shown in Figure 3, SQS offers a rather steady performance, whereas HBaseMQ gets saturated as the number of concurrent clients increases. This test verifies that there is an upper limit of scalability for HBaseMQ due to the design of using a single queue table for all clients in the queue. But for a moderate number of clients (about 200) per queue, HBaseMQ still outperforms SQS.

V. ACKNOWLEDGEMENT

This work was supported in part by the NSERC Discovery Grant 341823, and US National Science Foundation Award 1116606.

VI. CONCLUSIONS AND FUTURE WORK

This paper presents HBaseMQ, the first distributed message queuing system based on bare-bones HBase. HBaseMQ targets at existing HBase users to easily enable message queuing with minimum effort. HBaseMQ supports reliable and total order message delivery with “at least once” or “at most once” message delivery guarantees at user's choice. Additionally, operations on messages satisfy database level ACID requirements. These features are normally only available in full-fledged messaging systems that require installation and maintenance of a standalone messaging system. Future work includes improving the scalability of the system in terms of adding splittable queue tables, explore send-only and receive-only workloads and then a variety of send/receive ratios, and further stress test the system on larger scale deployment.

REFERENCES

- [1] B. Snyder, D. Bosanac, and R. Davies. ActiveMQ in Action. Manning Publication, ISBN: 1933988940, March, 2011.
- [2] S. Vinoski, Advanced Message Queuing Protocol, IEEE Internet Computing, 10(6): 87-89, 2006.
- [3] R. T. Fielding. Architectural styles and the design of network-based software architectures. Ph.D. dissertation, University of California, Irvine, 2000.
- [4] P. Saint-Andre. Extensible Messaging and Presence Protocol (XMPP): Core. RFC 3920 (Proposed Standard), Internet Engineering Task Force, Oct. 2004.
- [5] The Underlying Technology of Messages, <http://www.facebook.com/notes/facebook-engineering/the-underlying-technology-of-messages/454991608919>
- [6] Hadoop, <http://hadoop.apache.org/>
- [7] HBase, <http://hbase.apache.org/>
- [8] IBM WebSphere MQ, <http://www-01.ibm.com/software/integration/wmq/>
- [9] Microsoft MSMQ, <http://msdn.microsoft.com/en-us/library/windows/desktop/ms711472>
- [10] Oracle Advanced Queuing, <http://docs.oracle.com/cd/B1050001/appdev.920/a96587/qintro.htm>
- [11] D. Peng and F. Dabek. Large-scale incremental processing using distributed transactions and notifications, Proceedings of the 9th USENIX conference on Operating systems design and implementation, OSDI10, Berkeley, CA, USA, 2010.
- [12] A. Videla and J. J.W. Williams. RabbitMQ in Action. ISBN: 9781935182979, Manning Publications, publisher's official E-book, Soft-bound print: April 2012.
- [13] Amazon Simple Queue Service (SQS), <http://aws.amazon.com/sqs/>